

Zero-Knowledge Proofs (ZKPs)

DSC 4110/6111: Modern Cryptography and AI/ML Security

Dr. Laltu Sardar

School of Data Science,
Indian Institute of Science Education and Research Thiruvananthapuram (IISER TVM)



September 28, 2026

Table of Content

- 1 Zero-Knowledge Proofs
- 2 Zero-Knowledge Proofs
- 3 ZKP for Machine Learning as a Service
- 4 Example: Proving Knowledge of a Secret Key
- 5 ElGamal and Schnorr Protocol
- 6 From Interactive to Non-Interactive ZKP
- 7 Towards Range Proofs

Zero-Knowledge Proofs

Zero-Knowledge Proof: Motivation

A *Zero-Knowledge Proof (ZKP)* allows a prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ that a statement is true without revealing the underlying secret information.

Basic Setting

- $\mathcal{P}$ knows a secret witness w .
- $\mathcal{P}$ wants to prove that w satisfies some relation.
- $\mathcal{V}$ knows the public statement x .
- The verifier should learn nothing beyond the validity of the statement.

$$R(x, w) = 1$$

Prove knowledge of w without revealing w

What Does a ZKP Provide?

A ZKP is usually characterized by three properties.

Completeness

If the statement is true and the prover follows the protocol, the verifier accepts with overwhelming probability.

Soundness

A cheating prover should not be able to convince the verifier that a false statement is true, except with negligible probability.

Zero-Knowledge

The verifier learns nothing about the witness beyond the fact that the statement is true.

$$\Pr[\mathcal{V} \text{ accepts} \mid (x, w) \in R] \approx 1$$

$$\Pr[\mathcal{V} \text{ accepts a false statement}] \leq \text{negl}(\lambda)$$

Example: Sudoku as a ZKP Problem

Suppose Alice claims that she knows a valid solution to a Sudoku puzzle.

Goal

Alice wants to convince Bob:

“I know a valid Sudoku solution.”

without revealing the solution itself.

- Public input: the Sudoku puzzle.
- Secret witness: a completed valid Sudoku grid.
- Verification: the completed grid satisfies all Sudoku constraints.

$$R(\text{Puzzle}, \text{Solution}) = 1$$

Question: Can Alice convince Bob without showing the solution?

Sudoku: The ZKP Idea

A simple conceptual solution is to hide the Sudoku solution using random permutations and commitments.

- 1 Alice prepares her valid solution.
- 2 She randomly permutes the symbols and positions.
- 3 She commits to the resulting hidden grid.
- 4 Bob asks random questions about the committed grid.
- 5 Alice opens only the cells required by the challenge.

Intuition

Repeated random challenges make cheating difficult, while the commitments prevent Alice from changing her answers later.

$$\Pr[\text{cheating succeeds in one round}] < 1$$

$$\Pr[\text{cheating succeeds in } t \text{ rounds}] \leq (\Pr[\text{cheating in one round}])^t$$

A New Problem: Machine Learning as a Service

Consider a machine-learning service operated by a server $\mathcal{S}$.

- Client sends an input x .
- Server evaluates a model f .
- Client does *not* want to receive the complete prediction.
- Server only reveals whether the result belongs to a specified range.

For example:

$$f(x) \in [L, U] \quad \text{or} \quad f(x) \notin [L, U].$$

New Question

How can the server convince the client that the claimed range condition is correct without revealing unnecessary information?

ZKP can prove the correctness of the hidden computation.

What Information Is Public and Secret?

Before selecting a ZKP protocol, we must identify the different types of data involved.

Public Information

- Model description or commitment to the model.
- Input, if it is public.
- Required output range $[L, U]$.
- Security parameters.

Private Information

- Private input x .
- Model parameters.
- Exact model output $y = f(x)$.
- Intermediate computation values.

Public statement x_{pub}

Private witness w

The Importance of the Data and Query

The required ZKP construction depends strongly on what must remain private.

Possible Requirements

- Private input, public model.
- Public input, private model.
- Private input and private model.
- Exact output hidden.
- Only a predicate about the output revealed.

For the range-query setting:

$$y = f(x)$$

but the verifier should learn only:

$$y \in [L, U].$$

Thus the ZKP statement can be expressed as:

$$\exists w : R(x_{\text{pub}}, w) = 1$$

where w contains the hidden input, computation, output, or other required secret information.

The statement determines the appropriate ZKP protocol.

Problem: Proving Possession of a Secret Key

Suppose a public-key encryption scheme has

$$(pk, sk) \leftarrow \text{KeyGen}(1^\lambda).$$

A prover $\mathcal{P}$ claims to know the secret key sk corresponding to the public key pk .

Goal

Prove:

$$\exists sk : \quad pk = \text{KeyGen}_{\text{pub}}(sk)$$

without revealing sk .

Question: Can the verifier test knowledge of sk without seeing sk ?

Challenge-Response Construction

Assume an encryption scheme (Enc, Dec) with public key pk .

1 Challenger chooses a random value r .

2 Challenger computes

$$c \leftarrow \text{Enc}(pk, r).$$

3 Challenger sends c to the prover.

4 Prover computes

$$r' \leftarrow \text{Dec}(sk, c).$$

5 Prover sends r' to the challenger.

6 Challenger accepts if

$$r' = r.$$

Intuition

Only someone possessing the corresponding secret key should be able to recover the random challenge r .

What Does This Protocol Prove?

The prover does not send the secret key.

Instead, it demonstrates its ability to perform a computation that requires the secret key.

$$c = \text{Enc}(pk, r)$$

$$r' = \text{Dec}(sk, c)$$

The verifier checks:

$$r' = r.$$

Important Observation

The challenge r is freshly generated and unknown to the prover before receiving the ciphertext.

Therefore, successful responses provide evidence that the prover possesses the corresponding secret key.

Knowledge of $sk \implies$ ability to answer fresh challenges

ElGamal Encryption

Let $\mathbb{G}$ be a cyclic group of prime order q with generator g .

The secret key is:

$$x \leftarrow \mathbb{Z}_q.$$

The public key is:

$$y = g^x.$$

To encrypt a message m , choose random $k \leftarrow \mathbb{Z}_q$ and compute:

$$(c_1, c_2) = (g^k, m y^k).$$

Decryption uses x :

$$m = \frac{c_2}{c_1^x}.$$

Question

Can we prove knowledge of x corresponding to the public key $y = g^x$ without revealing x ?

Schnorr Identification Protocol

The prover knows x such that:

$$y = g^x.$$

1 Prover chooses

$$r \leftarrow \mathbb{Z}_q$$

and sends

$$a = g^r.$$

2 Verifier chooses a random challenge

$$c \leftarrow \mathbb{Z}_q.$$

3 Prover computes

$$z = r + cx \pmod{q}$$

and sends z .

4 Verifier checks

$$g^z \stackrel{?}{=} a y^c.$$

Why Does Schnorr Work?

The honest prover sends:

$$z = r + cx.$$

Therefore:

$$g^z = g^{r+cx} = g^r (g^x)^c = a y^c.$$

Hence the verifier accepts.

What Is Hidden?

The prover never reveals x .

The verifier sees only:

$$(a, c, z).$$

Underlying Hardness

A verifier should not be able to compute x from

$$y = g^x$$

because this would require solving the discrete logarithm problem in the underlying group.

Schnorr as a Proof of Knowledge

The Schnorr protocol proves knowledge of a witness x satisfying:

$$y = g^x.$$

The relation is:

$$R(y, x) = 1 \iff y = g^x.$$

Thus:

Prover knows x such that $y = g^x$

without revealing x .

Key Idea

The verifier challenges the prover after seeing a commitment. The response must be consistent with both the commitment and the hidden witness.

Interactive vs. Non-Interactive Proof

The Schnorr protocol is interactive:

$$\mathcal{P} \xrightarrow{a} \mathcal{V} \xrightarrow{c} \mathcal{P} \xrightarrow{z} \mathcal{V}.$$

The verifier must generate and send the challenge c .

Goal

Remove the interaction while preserving the security properties.

A common approach is to derive the challenge using a hash function:

$$c = H(y, a, m),$$

where m may represent a message or protocol context.

The proof becomes:

$$\pi = (a, z).$$

Fiat-Shamir Transformation

Starting from the Schnorr protocol:

$$a = g^r, \quad c \leftarrow \mathbb{Z}_q, \quad z = r + cx.$$

Replace the random challenge with:

$$c = H(y, a).$$

The prover computes:

$$z = r + H(y, a)x \pmod{q}.$$

The verifier recomputes:

$$c = H(y, a)$$

and checks:

$$g^z \stackrel{?}{=} a y^c.$$

Interaction removed using a hash-derived challenge

Why Non-Interactive Proofs?

Non-interactive proofs are useful when the proof must be:

- generated once by the prover,
- transmitted to another party,
- verified later,
- verified by multiple parties,
- used inside a larger cryptographic system.

The proof can be represented as:

$$\pi = \text{Prove}(x, w)$$

and verification as:

$$\text{Verify}(x, \pi) \in \{0, 1\}.$$

Important Transition

We have moved from

interactive challenge-response

to

a transferable proof.

Next Problem: Proving a Range

Return to the machine-learning example.

Suppose the server computes:

$$y = f(x).$$

The server should convince the client that:

$$L \leq y \leq U$$

without revealing the exact value of y .

Desired Proof

The server proves:

$$\exists y, w : \quad y = f(x, w) \quad \wedge \quad L \leq y \leq U.$$

The verifier learns only that the condition is satisfied.

Can we prove that a hidden value lies in a public range?

This leads to Range Proofs.

Sources:

- 1 Apostol Vassilev, Alina Oprea, Alie Fordyce, Hyrum Anderson, Xander Davies, and Maia Hamin. *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*. NIST, 2025. NIST AI 100-2e2025. DOI: 10.6028/NIST.AI.100-2e2025.
- 2 Maria Rigaki and Sebastian Garcia. *A Survey of Privacy Attacks in Machine Learning*. ACM Computing Surveys, 56(4):101–134, 2024. DOI: 10.1145/3624010.

Acknowledgment:

- ChatGPT (OpenAI) was used to identify the content for the slides, which was subsequently verified by me.



Dr. Laltu Sardar, Assistant Professor, IISER Thiruvananthapuram
laltu.sardar@iisertvm.ac.in, laltu.sardar.crypto@gmail.com
Course webpage: https://laltu-sardar.github.io/courses/26_crypto.html.